

Session 12: A Deep Dive into Java's Memory Management

This session explores the crucial concept of garbage collection in Java, a fundamental process that automates memory management, freeing developers from the manual and error-prone task of memory allocation and deallocation. We will delve into the mechanics of garbage collection, how to suggest its execution, various techniques to make objects eligible for cleanup, and the role of the finalize method.

Garbage Collection in Java: The Automatic Memory Manager

In Java, the Java Virtual Machine (JVM) is responsible for managing memory. The primary area where objects are stored is called the **heap**. As a program executes, it creates objects on the heap. Over time, some of these objects are no longer needed. Garbage collection is the background process within the JVM that automatically identifies and reclaims the memory occupied by these unused objects, making it available for new object allocation.

The core principle behind garbage collection is **reachability**. An object is considered "live" as long as it is reachable from a set of "GC Roots." GC Roots are special variables that are always accessible, such as:

- **Local variables** in the currently executing method.
- **Active threads**.
- **Static variables** of a class.
- **JNI (Java Native Interface) references**.

Any object that is not reachable through any chain of references from a GC Root is considered "garbage" and is eligible for collection.

The most common garbage collection strategy is the **Mark-and-Sweep algorithm**:

1. **Mark Phase:** The garbage collector starts from the GC Roots and traverses the entire object graph, marking all reachable objects as "live."
2. **Sweep Phase:** The garbage collector scans the heap and reclaims the memory occupied by unmarked objects.

To optimize this process, the JVM's heap is often divided into generations:

- **Young Generation:** This is where all new objects are initially allocated. Since most objects in a typical application have short lifespans, this area is garbage collected frequently in an event called a **minor garbage collection**.
- **Old (or Tenured) Generation:** Objects that survive a few minor garbage collections are promoted to the Old Generation. This area is collected less frequently through a **major garbage collection** or a **full garbage collection**, which is a more time-consuming process.

Requesting the JVM to Run Garbage Collection

While garbage collection is an automatic process, Java provides a way for developers to *request* that the JVM run the garbage collector. It is crucial to understand that this is a **request, not a command**. The JVM can choose to ignore this request if it deems it unnecessary.

There are two primary ways to make this request:

1. **System.gc():** This is a static method in the `java.lang.System` class.
2. **Runtime.getRuntime().gc():** This method can be called on the singleton `Runtime` object associated with the current Java application.

Example:

Java

```
public class GcRequestExample {  
    public static void main(String[] args) {  
        String str = new String("Hello");  
        str = null; // Making the object eligible for GC  
  
        // Requesting the JVM to run the garbage collector  
        System.gc();  
        // or  
        // Runtime.getRuntime().gc();  
  
        System.out.println("Garbage collection requested.");  
    }  
}
```

Important Note: Explicitly calling the garbage collector is generally discouraged in production code. The JVM's own heuristics for triggering garbage collection are usually more efficient. Overriding this can lead to unnecessary performance overhead. These calls are primarily useful for debugging and testing purposes.

Different Ways to Make an Object Eligible for Garbage Collection

For an object to be garbage collected, it must become unreachable. Here are the common ways to make an object eligible for collection:

1. Nulling a Reference Variable

When a reference variable that points to an object is set to null, and there are no other references to that object, the object becomes unreachable.

Example:

Java

```
public class NullingReference {
    public static void main(String[] args) {
        // 'student' is a reference to a Student object
        Student student = new Student("Alice");

        // Now, the Student object is reachable through the 'student' reference.
        System.out.println(student.getName());

        // By setting 'student' to null, the original Student object is no longer
        // reachable (assuming no other references exist).
        student = null;

        // The Student object created with the name "Alice" is now eligible for
        // garbage collection.
    }
}

class Student {
    private String name;

    public Student(String name) {
        this.name = name;
        System.out.println("Student object created: " + this.name);
    }

    public String getName() {
        return name;
    }

    @Override
    protected void finalize() throws Throwable {
```

```

        System.out.println("Student object finalized: " + this.name);
    }
}

```

2. Re-assigning a Reference Variable

When a reference variable is re-assigned to a new object, the original object it was pointing to becomes unreachable, provided there are no other references to it.

Example:

Java

```

public class ReassigningReference {
    public static void main(String[] args) {
        // 'car1' refers to a Car object with the name "Sedan"
        Car car1 = new Car("Sedan");

        // 'car1' now refers to a new Car object with the name "SUV"
        car1 = new Car("SUV");

        // The original Car object ("Sedan") is now unreachable and eligible for
        // garbage collection.
    }
}

class Car {
    private String name;

    public Car(String name) {
        this.name = name;
        System.out.println("Car object created: " + this.name);
    }

    @Override
    protected void finalize() throws Throwable {
        System.out.println("Car object finalized: " + this.name);
    }
}

```

3. Island of Isolation

This is a more complex scenario where a group of objects reference each other, but none of them are reachable from any external live reference (i.e., from the GC Roots). This creates a self-contained "island" of objects that are all unreachable from the rest of the application. Modern garbage collectors are intelligent enough to identify and collect such islands of isolated objects.

Example:

Java

```
public class IslandOfIsolation {
    IslandOfIsolation i;

    public static void main(String[] args) {
        IslandOfIsolation t1 = new IslandOfIsolation();
        IslandOfIsolation t2 = new IslandOfIsolation();

        // Object t1 references object t2
        t1.i = t2;

        // Object t2 references object t1
        t2.i = t1;

        // At this point, both t1 and t2 are reachable.

        // Now, we nullify the external references to t1 and t2.
        t1 = null;
        t2 = null;

        // Even though t1 and t2 still reference each other, they form an
        // "island of isolation." Since they are not reachable from any
        // GC Root, both objects are eligible for garbage collection.

        System.gc(); // Requesting garbage collection
    }

    @Override
    protected void finalize() throws Throwable {
```

```
System.out.println("Finalize method called for an object in the island of isolation.");  
}  
}
```

The finalize() Method

The `finalize()` method, which belongs to the `java.lang.Object` class, is a special method that the garbage collector calls on an object just before it is reclaimed. The primary purpose of this method was to perform cleanup activities, such as releasing system resources (like file handles or database connections) that the object was holding.

Key Characteristics of the finalize() Method:

- **Called by the Garbage Collector:** You should never call the `finalize()` method directly.
- **Called at Most Once:** The garbage collector will call `finalize()` on an object at most one time.
- **No Guarantee of Execution:** There is no guarantee that the `finalize()` method will ever be called. If the JVM exits before the garbage collector gets a chance to run, the method will not be invoked.
- **Deprecated Since Java 9:** The `finalize()` method has been **deprecated** since Java 9 and is marked for removal in future versions. Its use is strongly discouraged due to several drawbacks:
 - **Unpredictable:** The timing of its execution is not guaranteed.
 - **Performance Overhead:** It can significantly slow down garbage collection.
 - **Error-Prone:** Exceptions thrown within a `finalize()` method are ignored, which can leave objects in a corrupt state.
 - **Resource Leaks:** Relying on `finalize()` for resource cleanup is unreliable and can lead to resource leaks.

Modern Alternatives to finalize():

For resource cleanup, the recommended approach is to use explicit `close()` methods in conjunction with the try-with-resources statement (introduced in Java 7). This ensures that resources are closed in a timely and predictable manner.

Example of finalize() (for educational purposes only):

Java

```
public class FinalizeExample {  
    public static void main(String[] args) {
```

```
FinalizeExample obj = new FinalizeExample();  
obj = null; // Making the object eligible for GC  
System.gc(); // Requesting GC  
System.out.println("End of main method.");  
}  
  
@Override  
protected void finalize() throws Throwable {  
    System.out.println("Finalize method called.");  
    // In a real (pre-Java 9) scenario, this is where you might have  
    // released native resources.  
}  
}
```

In summary, while understanding the `finalize()` method is important from a historical and conceptual perspective, its use in modern Java development is a significant anti-pattern. Always prefer structured and explicit resource management techniques.